QA Report 10: Infrastructure, CI/CD & DevOps

Project: OneCluster UBN Platform **Report ID:** QA-RPT-010 **Sprint Period:** March 31 - April 9, 2026 **Prepared By:** QA Team **Date:** April 9, 2026 **Status:** PARTIAL PASS (Critical Infrastructure Gaps)

---

1. Executive Summary

This report covers quality assurance testing for the infrastructure, CI/CD pipelines, and DevOps practices across the OneCluster UBN platform. This sprint saw extraordinary infrastructure activity with Chijindu contributing 190 commits focused on CI/CD pipeline establishment and Ezekiel-byte contributing 55 commits for VPS migration — all within this single sprint period.

The platform has migrated from Cloudflare Containers to VPS-based deployment. Docker Hub build workflows have been configured across all 16 services with Slack notification integration for build success/failure events. CODEOWNERS files have been added for automated review request routing, and secrets have been stripped from appsettings.json configuration files.

However, the dev environment is in a degraded state: only 5 of 16 services are operational. The Partner Gateway is returning 502 errors (PEG-375), and 9 banking/integration services are completely unreachable (PEG-376). This infrastructure instability is the primary risk to the platform's progress.

---

2. Scope

Infrastructure Components

Component	Description
Docker Hub Workflows	Automated container image build and push pipelines
VPS Deployment	Virtual Private Server hosting (migrated from Cloudflare Containers)
Slack Notifications	Build status notifications to team channels
CODEOWNERS	GitHub code ownership for automated PR review assignments
Secrets Management	Environment variable injection replacing hardcoded secrets
Service Health Monitoring	Endpoint availability verification

Services Inventory (16 Total)

#	Service	Category
1	Admin Gateway	Gateway
2	Partner Gateway	Gateway
3	Admin Auth Service	Authentication
4	Partner Auth Service	Authentication
5	User Management Service	Core
6	Admin Portal (Frontend)	Frontend
7	Partner Portal (Frontend)	Frontend
8	KYB Service	Onboarding
9	KYC Service	Onboarding

10	API Catalog Service	Core
11	Billing Service	Core
12	Banking API	Integration
13	3rd Party Integration	Integration
14	Notification Service	Support
15	Audit Service	Support
16	Analytics Service	Support

---

3. Changes This Sprint

Total Commits: 245 (190 Chijindu + 55 Ezekiel-byte)

This is by far the highest commit volume of any area this sprint, reflecting the foundational nature of the infrastructure work.

CI/CD Pipeline Establishment (Chijindu — 190 commits)

Change	Description	Impact
Docker Hub workflows — all 16 services	GitHub Actions workflows for building and pushing Docker images to Docker Hub on merge to main	High — automated build pipeline
Slack build notifications	Webhook integration sending build success/failure notifications to #ubn-builds Slack channel	Medium — team visibility
CODEOWNERS files	Added across all repositories to auto-assign reviewers based on file paths	Low — process improvement
Secrets stripping	Removed hardcoded secrets from appsettings.json across all .NET services	High — security hardening
Build caching	Docker layer caching in GitHub Actions to reduce build times	Medium — efficiency
Multi-stage Dockerfiles	Optimized Dockerfiles with build and runtime stages for smaller images	Medium — deployment efficiency
Branch protection rules	Configured required reviews and status checks on main branches	Medium — code quality gates

VPS Migration (Ezekiel-byte — 55 commits)

Change	Description	Impact
VPS provisioning	Server setup, OS hardening, Docker installation	High — hosting foundation
Migration from Cloudflare Containers	All services moved from Cloudflare Container platform to self-managed VPS	High — hosting platform change
Docker Compose configurations	Service orchestration files for each deployment environment	High — deployment definitions
Nginx reverse proxy setup	Request routing configuration for all services	High — traffic routing
SSL/TLS certificate setup	Let's Encrypt certificates for all service domains	High — HTTPS enforcement

Firewall configuration	UFW rules for service port access control	High — network security
Monitoring agent setup	Basic uptime monitoring for service endpoints	Medium — observability

---

4. Test Results

4.1 Dev Environment Service Health

Environment: Dev VPS **Result:** 5 PASS / 1 FAIL / 9 UNREACHABLE / 1 N/A

Test ID	Service	Health Endpoint	Expected	Actual	Status	Notes
INF-001	Admin Gateway	/health	200 OK	200 OK	PASS	Responding correctly
INF-002	Admin Auth Service	/health	200 OK	200 OK	PASS	Authentication working
INF-003	User Management Service	/health	200 OK	200 OK	PASS	—
INF-004	Admin Portal (Frontend)	/	200 OK	200 OK	PASS	Page loads on dev-admin.onecluster.co
INF-005	Partner Portal (Frontend)	/	200 OK	200 OK	PASS	Page loads on dev-ubn-ui.onecluster.co
INF-006	Partner Gateway	/health	200 OK	502 Bad Gateway	FAIL	PEG-375 — CRITICAL
INF-007	Partner Auth Service	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-008	KYB Service	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-009	KYC Service	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-010	API Catalog Service	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-011	Billing Service	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-012	Banking API	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-013	3rd Party Integration	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-014	Notification Service	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable
INF-015	Audit Service	/health	200 OK	Connection refused	FAIL	PEG-376 — Unreachable

INF-016	Analytics Service	/health	200 OK	Connection timeout	FAIL	PEG-376 — Unreachable

4.2 CI/CD Pipeline Tests

Test ID	Test Case	Expected	Actual	Status	Notes
INF-017	Docker Hub workflow triggers on merge	Workflow runs on push to main	Workflow triggered	PASS	All 16 repos configured
INF-018	Docker image build completes	Image built without errors	Build successful	PASS	Multi-stage build working
INF-019	Docker image pushed to registry	Image available on Docker Hub	Image present with correct tag	PASS	Tagged with commit SHA and latest
INF-020	Slack notification — build success	Success message in #ubn-builds	Notification received	PASS	Includes repo name, branch, commit
INF-021	Slack notification — build failure	Failure message in #ubn-builds	Notification received	PASS	Includes error summary
INF-022	CODEOWNERS auto-assignment	PR reviewer auto-assigned based on file paths	Reviewer assigned	PASS	Tested across 3 repos
INF-023	Build caching — cache hit	Subsequent builds use cached layers	Cache hit, build time reduced 40%	PASS	~8 min -> ~5 min
INF-024	Branch protection — no direct push	Direct push to main blocked	Push rejected	PASS	Requires PR with approval
INF-025	Branch protection — required checks	Merge blocked without passing CI	Merge blocked correctly	PASS	—

4.3 Secrets Management Tests

Test ID	Test Case	Expected	Actual	Status	Notes
INF-026	appsettings.json — no secrets	No connection strings, API keys, passwords in config files	No secrets found	PASS	Grep scan across all repos
INF-027	Environment variable injection	Services read secrets from env vars	Configuration loaded correctly	PASS	Verified on running services
INF-028	Docker Compose — env file reference	Compose files reference .env (not inline secrets)	.env referenced, not committed	PASS	.env in .gitignore

---

5. Issues Found

Issue ID	Severity	Title	Status	Description

PEG-375	CRITICAL	Partner Gateway returning 502 in dev environment	Open	The Partner Gateway health endpoint returns 502 Bad Gateway. The Nginx reverse proxy is receiving connections but cannot reach the upstream Partner Gateway container. Possible causes: container not running, incorrect port mapping, Docker network misconfiguration, or environment variable misconfiguration preventing service startup.
PEG-376	CRITICAL	9 banking/integration services unreachable in dev	Open	Nine services (Partner Auth, KYB, KYC, API Catalog, Billing, Banking API, 3rd Party Integration, Notification, Audit) are completely unreachable — connections are refused at the network level. These services may not have been deployed to the VPS, may have crashed on startup due to missing environment variables or database connections, or may have incorrect firewall rules blocking their ports. Analytics Service specifically shows connection timeout rather than refused, suggesting a different root cause (possibly running but unresponsive).
PEG-384	Medium	No automated deployment pipeline (manual Docker pull)	Open	While CI/CD builds and pushes images to Docker Hub automatically, deployment to VPS still requires manual SSH access and `docker pull` + `docker compose up` commands. There is no continuous deployment trigger. This creates a gap between image availability and actual deployment.
PEG-386	Medium	No centralized logging	Open	Services running on VPS write logs to individual container stdout/stderr. There is no centralized log aggregation (e.g., ELK, Loki, or Cloudwatch equivalent). Debugging issues like PEG-375 and PEG-376 requires SSH access to the VPS and manual `docker logs` inspection per container.
PEG-387	Low	No health check auto-recovery	Open	When a service health check fails, there is no automated recovery mechanism (container restart, alerting). Docker Compose health checks with restart policies are not configured. Failed services remain down until manually restarted.

---

6. Dev Environment Status Summary

```
Service Status Overview (Dev Environment)
==========================================

  [UP]    Admin Gateway            ..............  HEALTHY
  [UP]    Admin Auth Service       ..............  HEALTHY
  [UP]    User Management Service  ..............  HEALTHY
  [UP]    Admin Portal (Frontend)  ..............  HEALTHY
  [UP]    Partner Portal (Frontend) .............  HEALTHY
  [502]   Partner Gateway          ..............  BAD GATEWAY (PEG-375)
  [DOWN]  Partner Auth Service     ..............  UNREACHABLE (PEG-376)
  [DOWN]  KYB Service              ..............  UNREACHABLE (PEG-376)
  [DOWN]  KYC Service              ..............  UNREACHABLE (PEG-376)
  [DOWN]  API Catalog Service      ..............  UNREACHABLE (PEG-376)
  [DOWN]  Billing Service          ..............  UNREACHABLE (PEG-376)
  [DOWN]  Banking API              ..............  UNREACHABLE (PEG-376)
  [DOWN]  3rd Party Integration    ..............  UNREACHABLE (PEG-376)
  [DOWN]  Notification Service     ..............  UNREACHABLE (PEG-376)
```

```
[DOWN]  Audit Service            ..............  UNREACHABLE (PEG-376)
[DOWN]  Analytics Service        ..............  UNREACHABLE (PEG-376)

Availability: 5/16 services (31.25%)
```

---

7. Recommendations

1. **PEG-375 & PEG-376 — Immediate Infrastructure Triage (CRITICAL):** Schedule an infrastructure war room to diagnose and restore the 11 non-functional services. Recommended triage steps:
   - SSH to VPS and run `docker ps -a` to check container states
   - Inspect logs of stopped/crashed containers with `docker logs <container>`
   - Verify Docker network connectivity between containers
   - Check environment variable injection for database connection strings
   - Verify firewall rules (UFW) allow inter-container and external traffic on required ports
   - Validate Nginx upstream configurations match actual container ports
2. **Continuous Deployment (PEG-384):** Implement a deployment trigger in GitHub Actions that SSHes to the VPS and runs `docker compose pull && docker compose up -d` after a successful image push. This closes the CI/CD loop and ensures deployed services match the latest built images. Consider using a deployment tool like Watchtower for automatic container updates.
3. **Centralized Logging (PEG-386):** Deploy a lightweight log aggregation stack. For VPS environments, Loki + Promtail + Grafana is recommended for its low resource footprint. This would have significantly accelerated diagnosis of PEG-375 and PEG-376.
4. **Health Check Auto-Recovery (PEG-387):** Add Docker Compose health check configurations with `restart: unless-stopped` or `restart: on-failure` policies. Example:

   ```yaml
   healthcheck:
     test: ["CMD", "curl", "-f", "http://localhost:80/health"]
     interval: 30s
     timeout: 10s
     retries: 3
     start_period: 40s
   restart: unless-stopped
   ```

5. **Infrastructure as Code:** The current VPS setup was performed manually (55 commits of iterative configuration). Document the server setup in an infrastructure-as-code format (Ansible playbook or shell scripts) to enable reproducible environment provisioning and disaster recovery.
6. **Monitoring & Alerting:** Implement uptime monitoring with alerting (e.g., UptimeRobot, Grafana alerts, or a custom health check cron) that notifies the team immediately when services go down. The current state where 9 services are unreachable should trigger automatic alerts, not wait for QA to discover it.
7. **Pre-Deployment Smoke Tests:** Add a post-deployment step to the CI/CD pipeline that runs basic health checks against all deployed services and reports failures to Slack. This would catch deployment issues like PEG-376 within minutes of deployment rather than during the next QA cycle.

End of Report